

Técnicas de Representación y Compresión en Arreglos Ordenados

Diego Corales · Benjamín Riquelme · Álvaro Guarda · Cristóbal Mansilla

Universidad Austral de Chile — Valdivia, Chile

I. Introducción y Contexto

Almacenar grandes volúmenes de datos de forma compacta y, al mismo tiempo, poder buscar en ellos con rapidez es uno de los problemas centrales del diseño de algoritmos. Rara vez se pueden optimizar ambas cosas a la vez: ganar espacio suele costar tiempo, y viceversa. Este proyecto estudia ese compromiso de manera empírica y teórica sobre un caso concreto, la representación de arreglos numéricos ordenados de gran tamaño.

La hipótesis de rendimiento que guía el trabajo es la siguiente: las técnicas de compresión avanzadas, en particular Partitioned Elias-Fano, deberían reducir el espacio en memoria de forma drástica, hasta situarlo por debajo del 10 % de lo que ocupa la representación explícita de referencia. A cambio, esperamos que la decodificación durante las búsquedas introduzca un costo adicional, constante o logarítmico. La pregunta de fondo es si ese sacrificio en tiempo se justifica, y sostenemos que sí lo hace en arquitecturas donde la memoria es el recurso escaso.

II. Metodología y Diseño Asintótico

A. Caso 1: Representación Explícita (Línea Base)

El primer caso es simplemente un arreglo contiguo de enteros sin signo de 64 bits (`vector<uint64_t>`), sin ningún tipo de compresión. Sobre él se ejecuta una búsqueda binaria iterativa. El único cuidado de implementación relevante es el cálculo del índice central, que se realiza evitando desbordamientos aritméticos al sumar los extremos del intervalo. Este caso sirve como punto de comparación tanto en espacio como en tiempo.

- **Complejidad de tiempo:** $O(\log n)$.
- **Complejidad de espacio:** $O(n)$ palabras de 64 bits, es decir 8 bytes por elemento.

B. Caso 2: Gap-Coding con Índice de Muestreo

En lugar de guardar cada valor, el segundo caso almacena las diferencias entre elementos consecutivos (los gaps). Como la secuencia está ordenada, estas diferencias son mucho más pequeñas que los valores originales y caben en tipos más acotados, en nuestro caso enteros de 32 bits. El problema es que ya no existe acceso aleatorio: para conocer el valor en una posición habría que sumar todos los gaps desde el origen. Para evitarlo, se construye un índice de muestreo que guarda el valor real cada b elementos, con $b = \sqrt{n}$ por defecto.

Así, una búsqueda se resuelve en dos etapas: primero una búsqueda binaria sobre el muestreo, que acota el intervalo donde podría estar el elemento, y luego una decodificación secuencial de los gaps dentro de ese intervalo.

- **Complejidad de tiempo:** $O(\log(n/b) + b)$. La búsqueda sobre el muestreo es logarítmica y la decodificación secuencial recorre a lo sumo b elementos.

- **Complejidad de espacio:** aproximadamente la mitad del Caso 1, al pasar de 8 a 4 bytes por elemento, más la sobrecarga reducida del índice.

Pseudocódigo de construcción y búsqueda:

```
// Construcción
GC[0] = A[0]
para i = 1 .. n-1:
    GC[i] = A[i] - A[i-1]
sample[j] = A[j*b]    para j = 0 .. n/b    // valores reales cada b

// Búsqueda de un valor x
1. Búsqueda binaria sobre 'sample' -> intervalo [L, R]
2. val = sample[L]
3. para i = L*b+1 .. R*b:    // decodificación secuencial
    val = val + GC[i]
    si val == x: devolver i
    si val > x: devolver 'no encontrado'
```

C. Caso 3: Partitioned Elias-Fano (PEF)

Elias-Fano clásico exige una secuencia monótonamente creciente, condición que el arreglo de gaps no cumple. Partitioned Elias-Fano resuelve esto dividiendo el arreglo en bloques de tamaño fijo ($B = 128$ en nuestra implementación) y acumulando los gaps localmente dentro de cada bloque, lo que produce una secuencia creciente sobre la cual sí es válido aplicar la técnica. El empaquetado se hace a nivel de bits, operando con máscaras y desplazamientos sobre palabras `uint64_t` tratadas como un mapa de bits continuo.

Pseudocódigo por bloque:

```
Para cada bloque de tamaño B:
    1. Acumular localmente: A_local[j] = A_local[j-1] + gap[j]
    2. U = A_local[B-1]    // máximo acumulado del bloque
    3. k = floor(log2(U / B))
    4. Para cada valor v en A_local:
        bits_bajos = v AND ((1 << k) - 1)
        bits_altos = v >> k
        escribir bits_bajos en k bits contiguos
        escribir bits_altos en unario (un 0 por incremento, un 1 de cierre)
```

III. Deducción Matemática del Espacio en PEF

PEF divide la representación de cada elemento en dos partes, una baja de longitud fija y una alta codificada en unario. Analizamos cada una por separado para estimar el costo total en bits por elemento.

- **Parte baja.** Todos los elementos del bloque reservan la misma cantidad de bits bajos, dada por $k = \lfloor \log_2(U/B) \rfloor$. Es un costo rígido y conocido de antemano para cada elemento.
- **Parte alta (unario).** El valor alto de cada elemento es el cociente $\lfloor v/2^k \rfloor$. Como la secuencia acumulada crece de forma estricta dentro del bloque, estos cocientes nunca decrecen. El número total de incrementos —los ceros que se escriben en unario a lo largo del bloque— es a lo sumo el cociente máximo, $U/2^k$. Por la definición de k se cumple $2^k \leq U/B$, de donde

$U/2^k \leq B$. Por lo tanto el bloque escribe como máximo B ceros en total; sumando un uno de cierre por cada uno de los B elementos, la parte alta ocupa a lo sumo 2B bits, es decir un promedio de aproximadamente 2 bits por elemento.

- **Espacio total proyectado.** Cada elemento requiere entonces, en teoría, alrededor de $k + 2$ bits. Esto desliga el costo de los tamaños fijos de 32 o 64 bits propios del hardware y reduce de forma considerable la huella de memoria.

IV. Resultados y Experimentación

Las pruebas se ejecutaron sobre arreglos de tamaño creciente, hasta $n = 10.000.000$ de elementos, midiendo el tiempo promedio de búsqueda y el espacio ocupado por cada estructura, para las distribuciones lineal y normal (esta última con desviación estándar 5). Se presentan ambos conjuntos de gráficos por separado.

A. Distribución Lineal

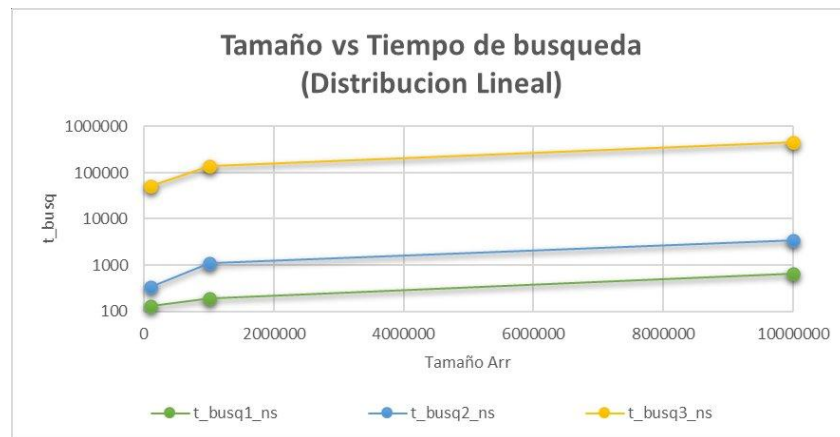


Fig. 1. Tamaño del arreglo (n) frente al tiempo de búsqueda (ns), distribución lineal. Eje vertical logarítmico.

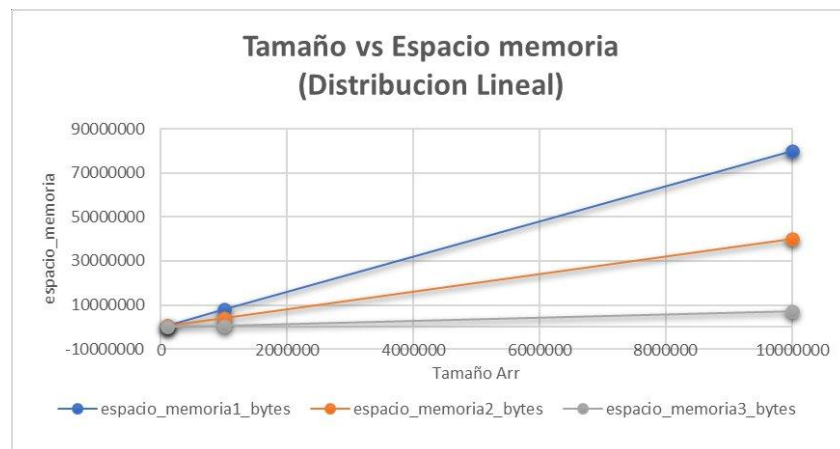


Fig. 2. Tamaño del arreglo (n) frente al espacio ocupado (bytes), distribución lineal.

En la Fig. 1 se observa la jerarquía de tiempos predicha por el análisis asintótico. Para $n = 10.000.000$, el Caso 1 resuelve la búsqueda en torno a 600 ns, el Caso 2 en unos 3.000 ns — cerca de cinco veces más— y PEF en aproximadamente 400.000 ns, casi tres órdenes de magnitud por encima de la línea base. Ese sobre costo refleja el trabajo de reconstruir la parte

unaria y aplicar las máscaras de bits en cada decodificación. La Fig. 2 confirma el análisis de espacio: el Caso 1 ocupa cerca de 80 MB (los 8 bytes por elemento esperados), el Caso 2 cae a unos 40 MB (justo la mitad) y PEF desciende hasta alrededor de 6,5 MB, menos del 10 % de la línea base (cercano al 8 %) y unos 5 bits por elemento, en línea directa con la estimación teórica de $k + 2$ bits de la Sección III.

B. Distribución Normal ($\sigma = 5$)

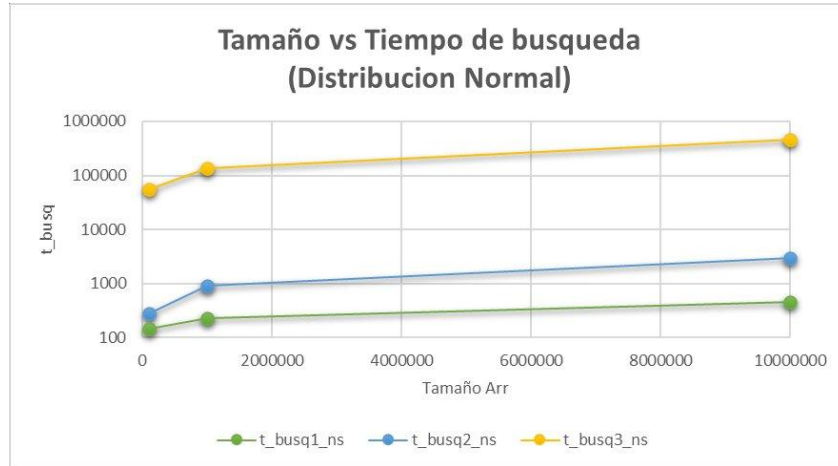


Fig. 3. Tamaño del arreglo (n) frente al tiempo de búsqueda (ns), distribución normal. Eje vertical logarítmico.

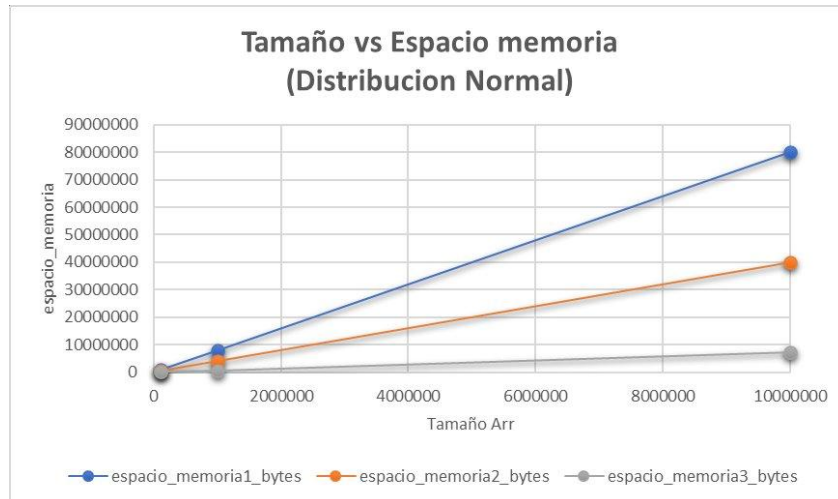


Fig. 4. Tamaño del arreglo (n) frente al espacio ocupado (bytes), distribución normal.

Los resultados con distribución normal mantienen exactamente la misma jerarquía y los mismos órdenes de magnitud que el caso lineal, tanto en tiempo como en espacio. Las únicas diferencias son leves: el tiempo del Caso 2 resulta marginalmente menor y el espacio de PEF se mantiene prácticamente idéntico. Esta coincidencia es esperable, porque en ambos escenarios los gaps generados son pequeños y de magnitud acotada, de modo que el parámetro k por bloque —y con él los bits por elemento que logra PEF— apenas varía entre una distribución y otra. En la práctica, esto sugiere que la eficiencia de la compresión depende mucho más de que los gaps sean pequeños que de la forma específica de la distribución que los origina.

Estructura	Costo espacial (teórico / empírico)	Costo temporal (búsqueda)
Caso 1 (Explícito)	100 % de referencia ($O(n)$)	$O(\log n)$
Caso 2 (Gap-Coding)	~50 % (gaps int32 + sample)	$O(\log(n/b) + b)$
Caso 3 (PEF)	<10 % (~8 %, $k+2$ bits/elem.)	$O(\log(n/b) + \text{decodificación PEF})$

V. Conclusiones

Los resultados respaldan la hipótesis planteada al inicio. Pasar de la representación explícita al Gap-Coding entrega una reducción de memoria cercana al 50 % a un costo prácticamente nulo, gracias al índice de muestreo que mantiene el tiempo de lectura acotado. Partitioned Elias-Fano lleva la compresión mucho más lejos: el comportamiento asintótico observado coincide con el análisis teórico y permite empaquetar los datos con un promedio de bits por elemento muy por debajo de un byte.

El precio de esa compresión es un mayor número de ciclos de CPU durante la búsqueda, necesarios para reconstruir la parte unaria y aplicar las máscaras de bits. Aun así, el ahorro de espacio —que lleva la huella de memoria a menos del 10 % de la línea base, en torno al 8 %— hace de PEF una opción adecuada en contextos donde el almacenamiento pesa más que unas fracciones de nanosegundo de latencia.

VI. Referencias

- [1] H. Ferrada, “INFO145: Diseño y Análisis de Algoritmos — Proyecto Semestral”, Universidad Austral de Chile, primer semestre 2026.
- [2] P. Elias, “Efficient storage and retrieval by content and address of static files”, *Journal of the ACM*, vol. 21, no. 2, pp. 246–260, 1974.
- [3] R. M. Fano, “On the number of bits required to implement an associative memory”, Memorandum 61, MIT Project MAC, 1971.
- [4] G. Ottaviano y R. Venturini, “Partitioned Elias-Fano indexes”, en *Proc. SIGIR*, pp. 273–282, 2014.
- [5] D. Salomon, *Data Compression: The Complete Reference*, 4ª ed., Springer, 2007.